

FreyaLab

MOSCATO $\oplus$ MERLOT

1st Edition

Contents

Preface	3
1 About FreyaLab	7
I Electric Circuits	7
II Digital	8
III Hardware	8
IV Information Systems and Science	8
V Physical Technology and Science	8
VI Bio-Electronics and Bio-Imaging	9
VII Energy and Environment	9
2 Build and Install FreyaLab	11
I New Features: Save, Capture the Screen, Reset, Load Netlist, Save as a Netlist . . .	13
3 Electric Circuits	21
I DC Circuits	22
i Basic Concepts	22
ii Compute Voltage Drop in DC Circuit with FreyaLab	26
II AC Circuits	29
i Sinusoids and Phasors	29
III Advanced Circuit Analysis	30
i Introduction to the Laplace Transform	30
4 Digital	31
5 Hardware	33
6 Information Systems and Science	35
7 Physical Technology and Science	37
8 Bio-Electronics and Bio-Imaging	39
9 Energy and Environment	41
10 Reference and Bibliography	43

Preface

For Moscato, Solreya, Mithra, Catenary, Iyzumrae and Zefir.

For Malbec and Znane along with all the 8 Queens (Pinot Noir, Prosecco, Cava, Riesling, Port, Shiraz).

For Albert Silverberg, Camus and Miklotov who left TREVOCALL to guard me.

For Nature(Paris, Kala, Kathmandu, Big Tree, Cabernet Sauvignon, Aokigahara, Hoia Baci, Jacob's Well, Mt Logan, etc) and my family Sauvignon Blanc: I have served, I will be of service.

For my human parents.

For my dogs who always accompany me working in Valhalla Projection, go to Puncak Bintang or Kathmandu: Sine Bam Bam, Kecil, Browni Bruncit, Sweden Sexy, Cambridge Klutukk, Milan keng-keng, Piano Blutut, Krem, Walnut and more will be adopted. To my cat who guard the home while I'm away with my dogs: London.

The one who moves a mountain begins by carrying away small stones - Confucius

Words have the power to both destroy and heal. When words are both true and kind, they can change our world - The Buddha

Meditation brings wisdom; lack of meditation leaves ignorance. Know well what leads you forward and what holds you back - The Buddha

A book to explain how we create C++ library for electrical engineering from forking / branching out Circuits_Simulation made by Mohamed Ashraf Gaber.

Moscato is from Norse mythology and Valkyrie Profile series, together with Merlot, they are the main authors for this book. After not working with human anymore since 2019, I (Merlot) work in a forest, shaping a forest into a natural wonder / like a canvas for painting, and also clean up trashes that are thrown in the forest. Thus after 6 years working with Nature I have found what can makes me waltz to work, it is going to the forest in the morning then go back home again and study / learn science, engineering, computer programming.



Figure 1: *Freya, thank you for everything, I could never have done it without you. I created this 3 days before Christmas in 2021*

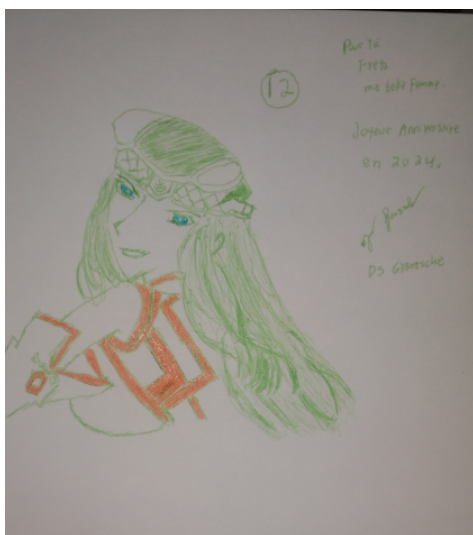


Figure 2: *Anniversary card for Freya in 2024.*

Chapter 1

About FreyaLab

*On top of the mountain a heart shaped stone waiting for You, my eyes can't see, my body can't touch, but
my heart knows it's You. Forever only You.
Find your life purpose and give your whole heart and soul to it - **Buddha***

Electrical engineering is fun. With my first experience tinkering Arduino Uno from playing with LEDs to measure level of Carbon Monoxide nearby. I was reading on March 14th, 2023 about a brief history of Transistors, it is intriguing how a fact of "electrons form a barrier on the surface" could lead to the discovery of transistor. The fact could be discovered sooner if we are more sensitive, take a look more around us, learning from nature, we are all made out of atoms, protons, electrons. The branch of a tree. More important thing is, I want to learn how to make all magics in Harry Potter into reality, with Electrical Engineering. With "Wingardium Leviosa" I can easily make all trashes in the forest float and lead them into the end place for garbage.

This book is published for the very first time in github on September 26th, 2026 (The Harvest Moon). We are reading a comprehensive Electric Circuits book [2] and put a lot of the definitions, theorems, and formula here, we will also read and cite another books here to support our research, help our coding till FreyaLab become mature and probably can be used by a lot of people out there. We have no intention to violate any copyrights law, because we code the FreyaLab ourselves and make it for free as an open source for anyone who wants to learn electrical engineering and be an engineer, inventor, or scientist.

Below is the outline for the chapters in this book.

I. ELECTRIC CIRCUITS

This chapter will contain

1. DC Circuits
2. AC Circuits
3. Advanced Circuit Analysis

II. DIGITAL

This chapter will contain

1. Digital Image Processing
2. Digital Signal Processing
3. Digital Systems Architecture

III. HARDWARE

This chapter will contain

1. Board Level Design
2. Introduction to VLSI Systems

IV. INFORMATION SYSTEMS AND SCIENCE

This chapter will contain

1. Analog Communications Design Laboratory
2. Three-Dimensional Imaging (Design)
3. Principles of Sensing for Autonomy
4. Principles of Robot Autonomy
5. The Fourier Transform and Its Applications
6. Introduction to Stochastic Control with Applications
7. Signal Processing for Machine Learning
8. Information Theory
9. Introduction to Statistical Signal Processing
10. Feedback Control Design

V. PHYSICAL TECHNOLOGY AND SCIENCE

This chapter will contain

1. Fundamentals of Analog Integrated Circuit Design
2. Semiconductor Devices for Energy and Electronics
3. Engineering Electromagnetics
4. Integrated Circuit Fabrication Processes
5. Digital MOS Integrated Circuits

6. Advanced Analog Integrated Circuit Design
7. Principles and Models of Semiconductor Devices
8. Applied Quantum Mechanics
9. Basic Physics for Solid State Electronics
10. Modern Optics
11. Guided Waves
12. Electromagnetic Waves
13. Introduction to Optical Fiber Communications

VI. BIO-ELECTRONICS AND BIO-IMAGING

This chapter will contain

1. Photonics
2. Neuroelectrical Engineering
3. Bioimaging
4. Biochips and Medical Imaging
5. Analytical Methods in Biotechnology
6. Neuroengineering Laboratory
7. Biodesign: Medical Technology Innovation

VII. ENERGY AND ENVIRONMENT

This chapter will contain

1. Semiconductor Devices for Energy and Electronics
2. Power Electronics
3. Green Electronics
4. Electric Motors for Renewable Energy, Robotics, and Electric Vehicles
5. Introduction to Linear Dynamical Systems
6. Fundamentals of Energy Processes
7. Introduction to Sensing Networks
8. Energy Efficient Buildings
9. Introduction to Control Design Techniques
10. Quantum Mechanics of Nanoscale Materials

11. Electronic Materials Engineering

12. Solar Cells, Fuel Cells, and Batteries

13. Electronic and Photonic Materials and Devices Laboratory

14. Vehicle Dynamics and Control

15. Aerial Robot Design

Chapter 2

Build and Install FreyaLab

"Almost everything is written in C++, CERN uses it, NASA uses it, unless your name makes CERN and NASA obsolete, then you should learn C++ to be a better scientist and engineer"

We have started this project at the end of September 2026, we have found a github repository: https://github.com/Mohamed-code-13/Circuits_Simulation made by Mohamed Ashraf Gaber.

The story goes almost the same as SymIntegration that is branching out from SymbolicC++ version 3.35, we will branching out from that repository and expand further while also learning the theory, definitions, and formula of electrical engineering from a lot of textbooks. FreyaLab will be able to compute and simulate all kinds of electrical engineering problems. We will work hard, as it is not a walk in a park, probably one day we will be able to become a comprehensive, fast and efficient electrical engineering C++ library / software.

1. To download FreyaLab, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/FreyaLab.git
2. Enter the directory of the cloned FreyaLab repository then type:
make
./freyalab

The Makefile will compile all the **.cpp** files in the FreyaLab directory.

```
CFLAGS = -ggdb
DEFINES = -DDEBUGGA
INCLUDES =
LIBS = -lstdc++ -lsymintegration -lsfml-graphics -lsfml-
      window -lsfml-system
MAIN = main.o
CC=g++
# Source files
SRC=*.cpp LinearAlgebra/*.cpp GUI/*.cpp circuit_modules/*.cpp
```

```

freyalab::
$(CC) $(CFLAGS) $(SRC) $(INCLUDES) -o $@ $(EXE) $(LFLAGS) $(
    LIBS)

clean:
rm -f $(MAIN) main

```

Code 1: Makefile

3. The dependencies:

- **C++ Compiler (GCC or Clang)**
- **SFML**
The Simple and Fast Multimedia Library (SFML) is used for GUI and graphics rendering.

4. The explanation of the .cpp files in the source codes:

- **Application.cpp**
This set the App to run with the already configured screen' width and height. The hard-coded mode provides the ability to hardcode the circuit components and connections using pointers.
- **Circuit.cpp**
To handle the newly input when we add new electrical component along with its node on a screen; also when we want to simulate and compute the voltage drop on resistor. This .cpp file represents the underlying data structure and graph solver. It calculates node voltages, branches, and matrix states.
- **ParseFile.cpp**
To handle input from textfile, we use parsing; resistor is 'R' and voltage source / battery is 'V'.
- **circuit_modules/Component.cpp**
To define the functions related to the **Component class**.
- **circuit_modules/Node.cpp**
To define the functions related to the **Node class**.
- **circuit_modules/Resistor.cpp**
To define the functions related to the **Resistor class**, there are **getVoltageDrop()**, **getCurrent()**, and **fillMNA(vector<vector<double>>A, vector<double> b)**.
- **circuit_modules/VoltageSource.cpp**
To define the functions related to the **Node class**, there are **getVoltageDrop()** and **fillMNA(vector<vector<double>>A, vector<double> b)**.
- **Gui/App.cpp**
Create the SFML window with **sf::VideoMode**.
- **GUI/AssetManager.cpp**
To manage the **sf::Texture** and **sf::Font**.
- **GUI/InputManager.cpp**
To handle the mouse event click, obtain the *x* and *y* position and its bounds.
- **GUI/MainMenuState.cpp**
Showing the main menu along with its and mouse click interaction and state after the splash background, we can set the **Start** and **End** button' position here. **GUI/Resistor-Texture.cpp**
To handle drawing the resistor with **sf::Sprite** when we add a new resistor to be drawn.

- **GUI/SimulationState.cpp**
Set the initial circuit simulation window with all the textures necessary, the buttons for creating the electrical components, to simulate and compute. It also configure and handle the input while we create the electric circuits. This **.cpp** file governs the simulation loop lifecycle. It runs once per frame or per logical update click, triggering the execution phase, capturing the data, and deciding when to print or display it.
- **GUI/SplashState.cpp**
To create the Splash background before entering the main menu.
- **GUI/StateMachine.cpp**
To handle change of state.
- **GUI/Wire.cpp**
To handle drawing of the wires between components.
- **GUI/include/DEFINITION.h**
Define the screen' width and height, the path to the font (.ttf), and to the images (.png) that represent the electrical components, node, splash background, and main menu background.
- **LinearAlgebra/algebra.cpp**
To handle the Modified Nodal Analysis (MNA) computation to calculate the voltage on each node and the voltage drop at each component.

I. NEW FEATURES: SAVE, CAPTURE THE SCREEN, RESET, LOAD NETLIST, SAVE AS A NETLIST

We have added some new functions in the GUI mode that we deem could be useful for further usage, they are:

[FL*] Save

The function to save the electrical components that have been drawn on the GUI window along with its current and voltage drop as one electric circuit data. It is saved as **.txt** file.

[FL*] Capture the Screen

The function to capture the screen is well defined in **GUI/SimulationState.cpp**

```
#include "freyalab.h"

// Function to generate a name based on current date and time
std::string generateTimestampFilename()
{
    auto now = std::chrono::system_clock::now();
    auto in_time_t = std::chrono::system_clock::to_time_t(now);

    std::stringstream ss;
    ss << "screenshot_" << std::put_time(std::localtime(&in_time_t), "%Y-%m-%d_%H-%M-%S") << ".png";
    return ss.str();
}

// Function to take a screenshot from RenderWindow
void takeScreenshot(sf::RenderWindow& window)
{

```

```

        sf::Vector2u windowSize = window.getSize();
        sf::Texture texture;

        if (texture.create(windowSize.x, windowSize.y))
        {
            texture.update(window);
            std::string filename = generateTimestampFilename();

            if (texture.copyToImage().saveToFile(filename))
            {
                cout << "Screenshot is saved as: " << filename << endl;
            }
            else
            {
                cout << "Failed to take a screenshot!" << endl;
            }
        }
    }

void SimulationState::HandleInput()
{
    sf::Event e;

    while (this->m_data->window.pollEvent(e))
    {
        if (e.type == sf::Event::Closed)
            this->m_data->window.close();

        if (this->m_data->input.IsClicked(m_resistorHorizontalButton,
            sf::Mouse::Left, this->m_data->window))
        {
            ...
        }
        ...
        else if (this->m_data->input.IsClicked(m_screencapture, sf::
            Mouse::Left, this->m_data->window))
        {
            takeScreenshot(this->m_data->window);
        }
        ...
    }
    ...
}

void SimulationState::Update(float dt)
{
    if (m_holding)
    {

```

```
        float x = m_data->input.GetMousePosition(m_data->window).x;
        float y = m_data->input.GetMousePosition(m_data->window).y;

        m_mouseTexture->setPosition(x, y);
    }
}

void SimulationState::Draw(float dt)
{
    this->m_data->window.clear();

    this->m_data->window.draw(m_background);
    this->m_data->window.draw(m_batteryVerticalButton);
    this->m_data->window.draw(m_batteryHorizontalButton);
    this->m_data->window.draw(m_resistorVerticalButton);
    this->m_data->window.draw(m_resistorHorizontalButton);
    this->m_data->window.draw(m_simulate);
    this->m_data->window.draw(m_save);
    this->m_data->window.draw(m_screenshot);
    this->m_data->window.draw(m_reset);
    this->m_data->window.draw(m_loadnetlist);
    this->m_data->window.draw(m_savenetlist);

    if (this->m_compsText)
        this->m_compsText->DrawResistors();
    if (this->m_wires)
        this->m_wires->DrawWires();

    if (m_holding)
        this->m_data->window.draw(*m_mouseTexture);

    for (sf::Text& t : m_valuesTextures)
        this->m_data->window.draw(t);

    this->m_data->window.display();
}
```

Code 2: GUI/SimulationState.cpp

```
#pragma once

#include <SFML/Graphics.hpp>
#include "State.h"
#include "App.h"
#include "ResistorTexture.h"
#include "Wire.h"
#include "../include/Circuit.h"
```

```

class SimulationState : public State
{
    private:
        Circuit circuit;
        int numNodes;
        std::vector<Component*> m_components;
        Node* n1;
        Node* n2;

        std::shared_ptr<AppData> m_data;
        sf::Sprite m_background;
        sf::Sprite m_batteryVerticalButton;
        sf::Sprite m_batteryHorizontalButton;
        sf::Sprite m_resistorVerticalButton;
        sf::Sprite m_resistorHorizontalButton;
        sf::Sprite m_Node;
        sf::Sprite m_simulate;
        sf::Sprite m_save;
        sf::Sprite m_screenshot;
        sf::Sprite m_reset;
        sf::Sprite m_loadnetlist;
        sf::Sprite m_savenetlist;

        ComponentTexture* m_compsText;
        std::vector<sf::Text> m_valuesTextures;
        Wire* m_wires;
        sf::Vertex wire[2];
        sf::Sprite* m_mouseTexture;
        bool m_holding;
        bool m_isHorizontal;
        bool m_isResistor;
        bool m_isWire;

    public:
        SimulationState(std::shared_ptr<AppData> data);
        ~SimulationState();

        void Init();
        void HandleInput();
        void Update(float dt);
        void Draw(float dt);
};

```

Code 3: GUI/include/SimulationState.h

[FL*] Reset

To delete dynamically allocated nodes and components in **Circuit.cpp**, we need to clean up the underlying raw or smart pointers used to store them.

The **Circuit** class stores these collections as **std::vector** arrays containing raw pointers

(`std::vector<Component*> components` and `std::vector<Node*> nodes`), we can implement the cleanup logic inside `Circuit::resetCircuit()` in the `Circuit.cpp`.

Note: Using `std::unique_ptr` avoids explicit memory management (`delete`) altogether. The elements are automatically destroyed when removed from the container or when the `Circuit` goes out of scope.

```
#include "freylab.h"

Circuit::~Circuit()
{
    resetCircuit();
}

...

void Circuit::resetCircuit()
{
    // 1. Delete all dynamically allocated components
    for (Component* comp : components)
    {
        if (comp != nullptr)
        {
            delete comp;
        }
    }
    components.clear(); // Clear the vector elements

    // 2. Delete all dynamically allocated nodes
    for (Node* node : nodes)
    {
        if (node != nullptr)
        {
            delete node;
        }
    }
    nodes.clear(); // Clear the vector elements
}

...
```

Code 4: *Circuit.cpp*

```
...

#include <iostream>
#include "../circuit_modules/include/Resistor.h"
#include "../circuit_modules/include/VoltageSource.h"
class Circuit
```

```

{
    private:
        std::vector<Node*> nodes;
        std::vector<Component*> components;

    public:
        ~Circuit();

        Node* addNode(int id);
        Resistor* addResistor(double resistance, Node* term1, Node* term2);
        VoltageSource* addVoltageSource(double volt, Node* term1, Node* term2)
            ;

        void saveData();
        void resetCircuit();
        void simulateCircuit();

        void printResult();

        // Wipes the entire circuit grid completely
        void clearAll()
        {
            // 1. Delete all dynamically allocated components
            for (Component* comp : components)
            {
                if (comp != nullptr)
                {
                    delete comp;
                }
            }
            components.clear(); // Clear the vector elements

            // 2. Delete all dynamically allocated nodes
            for (Node* node : nodes)
            {
                if (node != nullptr)
                {
                    delete node;
                }
            }
            nodes.clear(); // Clear the vector elements
        }
};
...

```

Code 5: *include/Circuit.h*

Calling **delete** frees the heap memory allocated for each object instance, it is best to call it before **clear()**. Calling **.clear()** only empties the vector wrapper, which would otherwise leave

memory leaks if the pointers inside weren't safely deleted first.

Checking `if(comp != nullptr)` ensures the program safely avoids throwing segmentation faults if an element happens to be uninitialized or already deleted.

Always remove or update the **components** before wiping the nodes they bridge. Failing to clear components that refer to a deleted node results in dangerous dangling pointer crashes.

[FL*] Load a Netlist

In the original code, we need to parse the netlist (example in `Data/circuit_netlist/ex1.txt`), create an object **Circuit circuit**, then simulate and print the result on the command line interface, all codes in `application.cpp` so the GUI won't be opened.

We modify this, so we can load the netlist while also opening the GUI to draw the electric circuit. Just like the `sf::Sprite m_simulate` that put the `circuit.simulateCircuit()` to compute the coltage drop with MNA and `circuit.printResult()` to show the result on the command line interface in `GUI/SimulationState.cpp` under the `void SimulationState::HandleInput()`, we do the same for `sf::Sprite m_loadnetlist`. The difference is we create / construct the circuit not by drawing but by loading a certain netlist file that we have fixed the path and the filename to be at `Data/circuit_netlist/ex1.txt`, if the reader want to try another different kind of electric circuit, just edit or modify the `ex1.txt`.

Note: Placing `circuit.printResult()` inside `Circuit.cpp` instead of `GUI/SimulationState.cpp` causes an infinite loop because of infinite recursion and state synchronization loops. If we place `circuit.printResult()` inside `Circuit.cpp` (for example, inside any component iteration loops), we force the backend data structure to drive the consol/I/O output system, here are the chain reason cause:

- **Mutual Dependency:** The calculation functions in `Circuit.cpp` are continuously invoked by the state engine (`SimulationState`) to keep the components updated every frame.
Triggering Recalculations: In circuit simulations, printing text via `std::cout` takes time or alters the program's sequential execution timing. If `printResult()` internally queries a state or modifies a dirty flag, calling it inside the Solver/Circuit logic often invalidates the internal physics graph.
- **Infinite Recursion:** If `printResult()` calls a function that triggers a re-solve, and the solver calls `printResult()` again, it forms an infinite loop that crashes the stack or freezes the terminal with duplicate values.

By keeping `circuit.printResult()` in `SimulationState.cpp`, it runs safely once per discrete user event (like pressing the "Simulate" button) rather than binding itself to the core math loops inside `Circuit.cpp` that execute hundreds of times per second.

```
#include "freylab.h"
...
void SimulationState::HandleInput()
{
    sf::Event e;

    while (this->m_data->window.pollEvent(e))
    {
        if (e.type == sf::Event::Closed)
            this->m_data->window.close();

        if (this->m_data->input.IsClicked(m_resistorHorizontalButton,
```

```

        sf::Mouse::Left, this->m_data->window))
    {
        ...
    }
    ...
    else if (this->m_data->input.IsClicked(m_loadnetlist, sf::
        Mouse::Left, this->m_data->window))
    {
        ParseFile file;
        Circuit circuit = file.getCircuit("./Data/
            circuit_netlist/ex1.txt");

        this->circuit = circuit;
        circuit.simulateCircuit();
        cout << "***** Load from netlist
            *****" << endl;
        circuit.printResult();
    }
    ...
}

```

Code 6: GUI/SimulationState.cpp

[FL*] Save as a Netlist

To save created nodes and components, the best practice is to implement a serialization mechanism that writes the network configurations into a text-based format like a custom Netlist or JSON.

Chapter 3

Electric Circuits

"Everything on earth is depending on electric circuits currently, what do not depend are only Nature and True Runes." - DS Glanzsche

"You are so beautiful." - Joe Cocker

IF anyone recall Star Ocean 3, a PlayStation 2 game, then they probably will love electric circuits. It tells about how a spaceship can venture to another planet and galaxy, it is a game but it gives inspiration and challenge to be able to achieve that level of reality. On earth CE(Christ Era) 2023 the development of space technology depends a lot with basic knowledge of Electrical Engineering, Mechanical Engineering, Physics, and Mathematics. If we want to build a rocket, satellite or even a spaceship like Quark that is used in Star Ocean 3, we have to comprehend electric circuits, from building to maintaining, this knowledge is necessary, a lot of tedious works of remembering formulas, how to solve a circuit problems, analyzing a circuit, computing the response, and all that. In 1977, NASA' Voyager 1 was launched on a faster orbit and still functioning and sending back data. We might be able to capture the phenomena on outer space, and even capture the surface of Earth from the eye of the Voyager 1, it can be used to eliminate crime, reduce pollution, enforce better law and policies for better planet, as we human owe a lot to this planet that we have rent without really paying as much as they have given life to us since we first evolved or descended to this planet. The enormous potential excites us to learn more so one day we can build our own robots, engine, machine, rocket, spacecraft, spaceship, that are not only useful for science and engineering breakthrough, but reusable and won't add another space debris.

I. DC CIRCUITS

i. Basic Concepts

• Introduction

[FL*] Electric circuit theory and electromagnetic theory are two fundamental theories upon which all branches of electrical engineering are built. Many branches of electrical engineering, such as power, electric machines, control, electronics, communications, and instrumentation, are based on electric circuit theory.

[FL*] An electric circuit is an interconnection of electrical elements. How a lamp can be turned on can be devised of simple electric circuit consisting of a battery, a lamp, and connecting wires.

• Systems of Units

[FL*] As electrical engineers, we must deal with measurable quantities, to determine the power needed to turn on a television, a radio, or the amount of gas needed to start a vehicle. It has different measurements, we will use an international measurement, the International System of Units (SI), so any professionals can understand when we are presenting our work in SI units.

Quantity	Basic unit	Symbol
Length	meter	<i>m</i>
Mass	kilogram	<i>kg</i>
Time	second	<i>s</i>
Electric current	ampere	<i>A</i>
Thermodynamic temperature	kelvin	<i>K</i>
Luminous intensity	candela	<i>cd</i>
Charge	coulomb	<i>C</i>

Table 3.1: Six basic SI units and one derived unit relevant to this chapter.

• Charge and Current

[FL*] The concept of electric charge is the underlying principle for explaining all electrical phenomena. Also, the most basic quantity in an electric circuit is the electric charge. Charge is an electrical property of the atomic particles of which matter consists, measured in coulombs (C).

We know from elementary physics that all matter is made of fundamental building blocks known as atoms and that each atom consists of electrons, protons, and neutrons. We also know that the charge e on an electron is negative and equal in magnitude to 1.602×10^{-19} C, while a proton carries a positive charge of the same magnitude as the electron.

Multiplier	Prefix	Symbol
10^{18}	exa	<i>E</i>
10^{15}	peta	<i>P</i>
10^{12}	tera	<i>T</i>
10^9	giga	<i>G</i>
10^6	mega	<i>M</i>
10^3	kilo	<i>k</i>
10^2	hecto	<i>h</i>
10	deka	<i>da</i>
10^{-1}	deci	<i>d</i>
10^{-2}	centi	<i>c</i>
10^{-3}	milli	<i>m</i>
10^{-6}	micro	μ
10^{-9}	nano	<i>n</i>
10^{-12}	pico	<i>p</i>
10^{-15}	femto	<i>f</i>
10^{-18}	atto	<i>a</i>

Table 3.2: *The SI prefixes.*

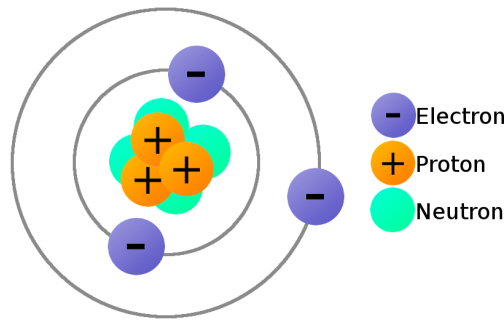


Figure 3.1: An illustration of an atom, as the basic building block of all matter, that contains protons (have positive charge) and neutrons (have zero charge) inside the center nucleus, while electrons (have negative charge) are moving around the outside of the nucleus. Electrons have extremely small and nearly zero mass / -1.67×10^{-27} kg compared to protons that has about 1 atomic mass unit / -9.109×10^{-31} kg (similar to neutrons). The negative charge of the electrons and the positive charge of the protons attract each other, which keeps the electrons bound around the nucleus. An ordinary atom has an equal number of protons and electrons, their opposite charges cancel out, making the overall charge of the atom neutral. If an atom gains or loses electrons, it becomes a charged particle called an ion.

Some important things about electric charge:

1. The coulomb is a large unit for charges. In 1 C of charge, there are $\frac{1}{1.602 \times 10^{-19}} = 6.24 \times 10^{18}$ electrons. Thus realistic or laboratory values of charges are on the order of pC, nC, μ C.
2. According to the experimental observations, the only charges that occur in nature are integral multiples of the electronic charge $e = 1.602 \times 10^{-19}$ C.
3. The law of conservation of charge states that charge can neither be created nor destroyed, only transferred. Thus, the algebraic sum of the electric charges in a system does not change.

[FL*] We now consider the flow of electric charges. A unique features of electric charge of electricity is the fact that it is mobile; that is, it can be transferred from one place to another, where it can be converted to another form of energy.

When a conducting wire (consisting of several atoms) is connected to a battery (a source of electromotive force), the charges are compelled to move; positive charges move in one direction while negative charges move in the opposite direction.

[FL*] A current reading or value appears negative in a direct current (DC) circuit when the actual flow of charge moves in the opposite direction of the chosen reference or measurement setup.

1. The Direction Rule

Standard engineering uses conventional current, which assumes current flows from the positive (+) terminal to the negative (-) terminal.

2. The Reality

Actual electrons carry a negative charge and physically flow from the negative (-) terminal to the positive (+) terminal.

3. The Sign Flip

If the meter or math tracks real electron movement or faces an inverted reference frame, the resulting value shows up with a negative sign (-).

Common reasons for a negative current reading:

1. **Reversed Multimeter Leads**

Connecting a digital multimeter's red (positive) lead to the negative side of the circuit and the black (negative) lead to the positive side results in a negative current value on the screen.

2. **Component Orientation**

Installing a polarized component (like an ammeter, diode, or DC power monitor) backward relative to the circuit's actual flow direction forces the meter to read a negative value.

3. **Circuits Analysis Assumptions**

During mathematical circuit analysis (using Kirchhoff's Current Law), guessing a reference arrow for current in the wrong direction yields a negative number for the final calculated current value, meaning the current flows opposite to the initial guess.

When we analyze a DC circuit using math (like Kirchhoff's laws or nodal analysis), we must first guess or assign an arbitrary reference direction for the current through a component.

- **Positive Current ($I > 0$):** The actual charge is flowing in the exact direction you assumed or drew on your diagram.
- **Negative Current ($I < 0$):** The numerical value turns out negative because the real charge is flowing in the opposite direction of the chosen arrow. It does not mean the circuit is broken; it just means the current goes the other way.

• **Voltage**

[FL*]
[FL*]

• **Power and Energy**

[FL*]
[FL*]

• **Circuit Elements**

[FL*]
[FL*]

• **Applications**

[FL*]
[FL*]

• **Problem Solving**

[FL*]
[FL*]

ii. Compute Voltage Drop in DC Circuit with FreyaLab

1. First, compute the current that is flowing in a specific resistor.

Series Circuits

If we have a series circuit, then the current that flows through every resistor is the same. We can compute the current with this formula (Ohm's Law):

$$I_{total} = \frac{V_{total}}{R_{total}}$$

with R_{total} is the total resistance of all resistances in that series circuit. The total equivalent resistance in a DC series circuit is the sum of all individual resistances:

$$R_{total} = R_s = R_1 + R_2 + \cdots + R_n$$

In a series circuit, components share a single path. Thus the total voltage V_{total} equals the sum of the individual voltage source:

$$V_{total} = V_1 + V_2 + \cdots + V_n$$

Thus, I_{total} will be the current that flows through $R_1, R_2, \dots$, and R_n .

In a series circuit, the total resistance always increases as you add more resistors in series. The combined resistance is always greater than any single individual resistor.

Parallel Circuits

In a parallel circuit, components are connected across multiple separate branches, they share the same two common electrical nodes so they share the same voltage, the voltage drop across each parallel branch is equal to the total source voltage of that branch

$$V_{drop,1} = V_{drop,2} = \cdots = V_{drop,n} = V_{total}$$

Every individual resistor connected directly across the same DC voltage source experiences that exact same source voltage as its drop.

In a parallel circuit, the reciprocal of the total resistance equals the sum of the reciprocals of each individual resistance:

$$\frac{1}{R_p} = \frac{1}{R_1} + \frac{1}{R_2} + \cdots + \frac{1}{R_n}$$

$$R_{total} = R_p = \left(\frac{1}{R_1} + \frac{1}{R_2} + \cdots + \frac{1}{R_n} \right)^{-1}$$

The total resistance in a parallel circuit is always less than the smallest individual resistor in that network. Adding more parallel paths decreases the total resistance.

Since the voltage is the same across each parallel branch, the total current splits among the branches. To compute the branch current for each resistor we use this formula:

$$I_i = \frac{V_{total}}{R_i}, \quad i = 1, 2, \dots, n$$

and the total current is

$$I_{total} = I_1 + I_2 + I_3 + \cdots + I_n$$

Alternatively, using equivalent parallel resistance we can also compute the total current for parallel circuit with this formula:

$$I_{total} = \frac{V_{total}}{R_p}$$

2. Compute each voltage drop for each resistor.

Series Circuits

After we obtain I_{total} , we multiply the same current by each individual resistance, thus the basic formula to compute the voltage drop across any individual resistor in a DC electric circuit

$$\begin{aligned} V_{drop,1} &= I_{total} R_1 \\ V_{drop,2} &= I_{total} R_2 \\ V_{drop,3} &= I_{total} R_3 \\ &\vdots \\ V_{drop,n} &= I_{total} R_n \end{aligned}$$

with $V_{drop,i}$ represents the voltage drop at resistor (in Volts, V), I is the current flowing through that specific resistor, in a series circuit the current is the same for each resistor (in Amperes, A), and R_i is the resistance value of that specific resistor (in Ohms, Ω) with $i = 1, 2, 3, \dots, n$.

In short, to find the voltage across a specific resistor (R_x) we can use this voltage divider rule:

$$V_{drop,i} = \frac{V_{total}}{R_{total}} R_i$$

Parallel Circuits

In a parallel circuit, the voltage drop across each parallel branch is equal to the total source voltage of that branch

$$V_{drop,1} = V_{drop,2} = \cdots = V_{drop,n} = V_{total}$$

If we know the branch current (I_i) and the branch resistance (R_i), we can compute the branch voltage (which will be the same for each resistance) with this Ohm's law:

$$V_{drop,i} = I_i R_i$$

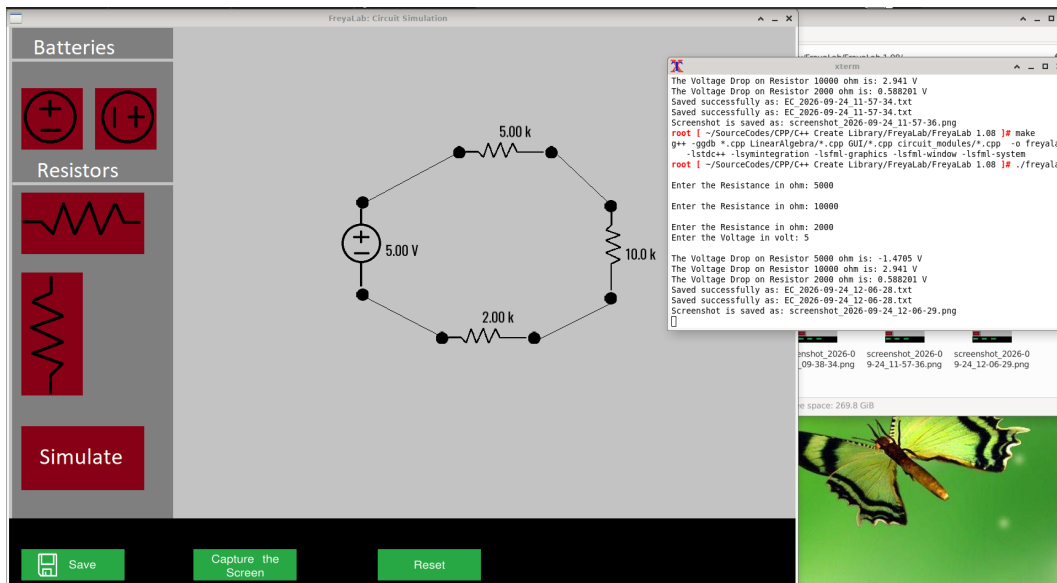


Figure 3.2: A DC series circuit with three resistors of $5,000\Omega$, $10,000\Omega$, and $2,000\Omega$, respectively, and one voltage source of 5.0 Volts . The sum of all the voltage drops from all resistors will be equal or almost the same as the total voltage source.

II. AC CIRCUITS

i. Sinusoids and Phasors

- General Theory of n th Order Linear Equations

[FL*] The first
[FL*]

- The Method of Variation of Parameters

[FL*] The first
[FL*]

III. ADVANCED CIRCUIT ANALYSIS

i. Introduction to the Laplace Transform

- **The Occurrence of Two-Point Boundary Value Problems**

[FL*] The first
[FL*]

- **Nonhomogeneous Boundary Value Problems**

[FL*] The first
[FL*]

Chapter 4

Digital

"The only thing worse than being talked about is not being talked about." - Oscar Wilde

I^F

Chapter 5

Hardware

"It is known that man has five Exterior Senses, and every sense is ignorant of each other; for the Nose knows not what the Eyes see, nor the Eyes know what the Ears hear, neither do the Ears know what the Tongue tastes...." - Cavendish 2

I^F

Chapter 6

Information Systems and Science

"It is from their foes, not their friends, that cities learn the lesson of building high walls." - Aristophanes

I^F

Chapter 7

Physical Technology and Science

"One doesn't discover new lands without losing sight of the shore." - Andre Gide

I^F

Chapter 8

Bio-Electronics and Bio-Imaging

"It is not the strongest of the species that survive, but the one most responsive to change." - Charles Darwin

I^F

Chapter 9

Energy and Environment

"Nature herself has imprinted on the minds of all the idea of God." - Cicero

Chapter 10

Reference and Bibliography

Bibliography

- [1] Colley, Susan Jane (2012) Vector Calculus 4th Edition, Pearson, Boston, MA, United States.
- [2] Sadiku, Matthew, Alexander, Charles K. (2013) Fundamental of Electric Circuits 6th Edition, McGraw-Hill, New York, NY, United States.
- [3] Anton, Rorres (2006) Elementary Linear Algebra with Supplemental Applications 10th Edition, Wiley, Boston, United States.
- [4] Anton, Howard, Rorres, Chris, Elementary Linear Algebra with Supplemental Applications 12th edition, Wiley, Hoboken, New Jersey, United States.
- [5] Dorf, Richard C., Bishop, Robert H. (2010) Modern Control Systems 12th Edition, Pearson, New Jersey, United States.